

A Lambda Data Pipeline for Ride-Hailing Fleet Operations

EC8202 Applied Big Data Engineering

Jackshan Venujan G.S

Dinojan V.

Register No. EG/2021/4566 Register No. EG/2021/4487

Summary. This project shows how a ride-hailing company can watch its fleet and review its daily costs. It uses two data paths. The live path gives a quick view of vehicle activity. The daily path uses the full day's vehicle data and a cost file to calculate each vehicle's income and profit or loss. The project uses Kafka, Spark, Airflow, PostgreSQL, an API, and monitoring tools. It runs with Docker Compose. One simulated day takes five real minutes.

Contents

1 Use Case and Business Need	2
2 Architecture Choice	2
2.1 Why two paths are needed	2
3 System Architecture	3
4 Technology Choices	3
5 Data Sources and Simulated Time	4
5.1 Live vehicle updates	4
5.2 Daily cost information	4
5.3 Clock used in the demo	4
6 Processing and Daily Calculations	4
6.1 Live processing	4
6.2 Daily batch processing	4
6.3 Daily calculations	5
7 Storage and Results for Users	5
8 Observability	5
8.1 Logs	5
8.2 Measures and alerts	5
9 Results	8
10 Limits and Future Improvements	8
11 Conclusion	8

1 Use Case and Business Need

A ride-hailing company needs to know how its vehicles are being used. Dispatchers need current information to decide where to send vehicles. The finance team needs to know whether each vehicle earned more than it cost to run.

These needs have different timing. The live view must update within seconds. It can be an estimate because new events keep arriving. The daily result must use the full day of data and the cost file. It is used to review vehicle performance, so the company needs a complete result.

Table 1: The two questions the system answers.

Need	Live fleet view	Daily review
Question	What are the vehicles doing now?	Which vehicles made or lost money today?
Data	Vehicle location, speed, status, and fares	Full vehicle history plus fuel and repair costs
When needed	Every few seconds	Once each simulated day
Result	Recent activity and alerts	Daily income, costs, and profit or loss
Accuracy	A quick estimate is useful	Use the complete available data

The system uses simulated vehicles and simulated costs. This lets the team show the full process without connecting to real drivers, cars, fuel providers, or garages. The two simulated sources describe the same set of vehicles.

2 Architecture Choice

The project uses a Lambda design. It has a live path and a daily path. Both paths use the same vehicle events, but they answer different questions.

2.1 Why two paths are needed

The live path reads new vehicle events and updates the current fleet view. Spark processes the events in small groups. This keeps the view fresh. Counts and earnings can be approximate because some events may arrive late and the day is not yet complete.

The daily path waits for a full day's vehicle history and the cost file. It then recalculates the day and joins vehicle income with fuel and repair costs. This gives the project its main daily financial result.

Table 2: Why the project uses Lambda instead of one stream-only path.

Point	Lambda design used here	One stream-only path
Live answer	A fast view is produced from incoming events	Can also produce a fast view
Daily cost file	The file is joined during the daily job	The file would need special handling in the stream
Late events	The full history can be read again	Events past the allowed wait time may be missed
Daily result	The day is recalculated with its costs	A live estimate is not enough for the final daily review

A batch-only system would not help dispatchers during the day. A stream-only system would not give the same full-day review after costs arrive. The two paths make the difference clear. The dashboard labels the live view as an estimate and the daily view as the full-day result.

There is a cost to this design. The system keeps a live path and a daily path. Some parts of the work are repeated. The daily path is still useful because the cost file arrives once a day and needs to be checked against the full vehicle history.

3 System Architecture

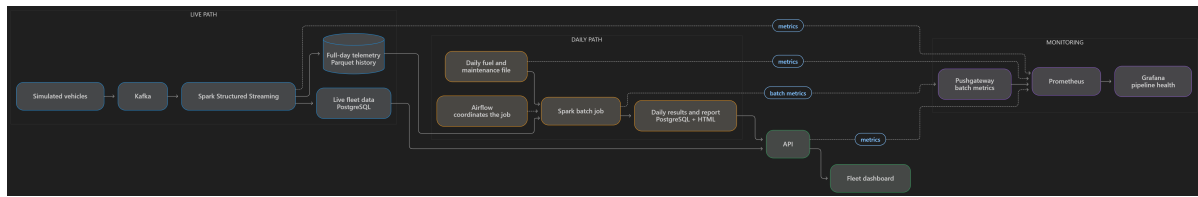


Figure 1: End-to-end architecture. Blue is the speed path, orange is the batch path, and green is storage. The master dataset is written by the speed layer and read again by the batch layer. This is the defining loop of a Lambda architecture.

The vehicle simulator sends updates to Kafka. Spark Structured Streaming checks the events and updates the live data in PostgreSQL. It also saves the full event history in Parquet. Invalid events are kept for review, and the system creates alerts when a vehicle stays idle too long.

The cost simulator makes one file for each simulated day. Airflow waits until the file and the matching vehicle history are ready. It then starts a Spark batch job and checks the result. The batch job writes the daily result to PostgreSQL and creates report files. The API reads the data and supplies it to the fleet dashboard.

Prometheus collects health measures from the long-running services. The short batch job sends its measures through Pushgateway. Grafana displays the measures and alerts.

4 Technology Choices

The tools were chosen to match the two data sources and the work they must do.

Table 3: Tools and why they fit this fleet system.

Part	Tool	Reason for using it
Vehicle data	Kafka	It receives and holds the stream of vehicle updates. Spark can read from it while the simulator keeps sending events.
Live processing	Spark Structured Streaming	It checks incoming events, makes recent summaries, updates vehicle status, and raises idle alerts.
Daily processing	Spark batch	It reads the complete day of vehicle history and joins it with the daily cost file.
Daily coordination	Airflow	It waits for both inputs, starts the batch job, checks the result, and records that the day is complete.
History storage	Parquet	It stores a large history in a compact form. The batch job can read the data for one day.
Serving storage	PostgreSQL	It stores the live and daily results in tables that the API can query.
Dashboard API	FastAPI	It provides a simple way for the dashboard and other users to request results.
Monitoring	Prometheus and Grafana	Prometheus gathers health measures and checks alert rules. Grafana displays the pipeline status.
Local setup	Docker Compose	It starts the services together and keeps saved data in named volumes between ordinary stops.

Spark is used for both live and daily work. This keeps the main data processing in one tool. The live job uses event time windows and can give a quick estimate. The daily job reads the full day again and produces the daily result. Airflow coordinates the batch work, but Spark does the data calculations.

5 Data Sources and Simulated Time

5.1 Live vehicle updates

The vehicle simulator represents twelve vehicles. Every two seconds it sends a vehicle update with a location, speed, status, time, and fare. A vehicle can be idle, travelling to a passenger, or carrying a passenger.

Some events are made invalid on purpose. Examples include a missing location, an impossible speed, an unknown status, or a fare increase while the vehicle is parked. Spark checks each event. Invalid events are kept apart so they can be reviewed instead of being used as good data.

5.2 Daily cost information

A second simulator creates one file for each simulated day. Each row may contain a vehicle's fuel cost, maintenance cost, distance, and service note. Some vehicles are left out. Some reported distances do not match the GPS distance. The daily job keeps these cases in the result and adds a flag so they are visible.

5.3 Clock used in the demo

One simulated day takes five real minutes. The calendar uses Sri Lankan time. This speed lets the team show a daily process in a short demo. A simulated hour takes about twelve and a half seconds. The live job uses 10-second and 15-second processing intervals. The daily job runs about once every five real minutes, after its inputs are ready.

The shared clock starts from one saved time so that all services use the same simulated date. This prevents the vehicle source, cost source, and daily job from using different day boundaries.

6 Processing and Daily Calculations

6.1 Live processing

Kafka carries the vehicle messages to Spark Structured Streaming. Spark checks the message shape and values, then adds a zone based on the vehicle location. It writes each event to the history, updates the latest vehicle state, and sends invalid events to a separate place for review.

A second streaming task groups activity by zone and time window. It calculates active vehicle counts, idle ratio, trips, earnings, and average speed. Counts are estimates because the live job uses an approximate counting method and only sees the events received so far. Late events can also fall outside the wait period.

An alert is made when a vehicle stays idle beyond the chosen limit. The system keeps the start of the idle period so it does not create a new alert for the same idle period every time it checks.

6.2 Daily batch processing

Airflow looks for a new daily cost file and waits for the matching vehicle history. It starts a Spark batch job once both inputs are available. The job reads all of the events for that simulated day and the costs in the file.

The daily job calculates the final fare for each completed trip. It also calculates distance from GPS points and estimates how much of the day each vehicle was working. It joins these figures with the fuel and maintenance costs.

For each vehicle, the report shows income, costs, profit or loss, profit margin, distance, and utilisation. The report can also show a missing cost record or a large difference between GPS distance and the reported distance. Airflow checks that the result was created before marking the day complete.

6.3 Daily calculations

Table 4: Main measures in the daily report.

Measure	How it is found
Income	Add the final fare for each completed trip.
Distance	Estimate distance from consecutive GPS points. Ignore jumps that are too large to be real.
Utilisation	Find the share of vehicle updates that were not idle.
Total cost	Add the fuel and maintenance costs from the daily file.
Profit or loss	Subtract total cost from income.
Flags	Mark low profit, missing cost data, or a large difference in reported and GPS distance.

The daily result is the main report for a completed simulated day. The live view is useful during the day, but it can be different because it is based on recent windows and partial data.

7 Storage and Results for Users

PostgreSQL holds the latest live vehicle information, recent zone summaries, alerts, and daily profit results. The Parquet history keeps the full set of vehicle events. The daily report is also saved as a CSV file and a web page.

The API reads the live and daily data and makes it available to the dashboard. The dashboard shows which view is live and which view comes from the completed day. It also compares the two so users can see how much the fast estimate differs from the daily result.

8 Observability

The project records structured logs, measures service health, and raises alerts. These features help the team find out whether data is arriving, whether Spark is processing it, whether a daily run finished, and whether the API can reach its database.

8.1 Logs

The services write structured log messages. Each message includes a time, service name, work stage, event, and useful details. This makes it easier to follow an event from the source through processing and storage.

8.2 Measures and alerts

Prometheus collects measures from the long-running services. The daily Spark job is short, so it sends its measures to Pushgateway. Prometheus then reads those measures.

The measures cover incoming event rate, last event time, invalid event rate, Spark processing time, processing delay, daily batch rows, API speed, and the age of the live and daily results. The project includes fifteen alert rules. Examples include no recent vehicle events, a stopped Spark job, too many invalid events, a late daily result, and a daily loss. A daily loss is a business alert. It does not mean the monitoring system has failed.

Grafana shows the main measures in the Fleet Pipeline Health dashboard. A health check also confirms that the API can reach PostgreSQL. Docker Compose uses service health checks to control start-up order.

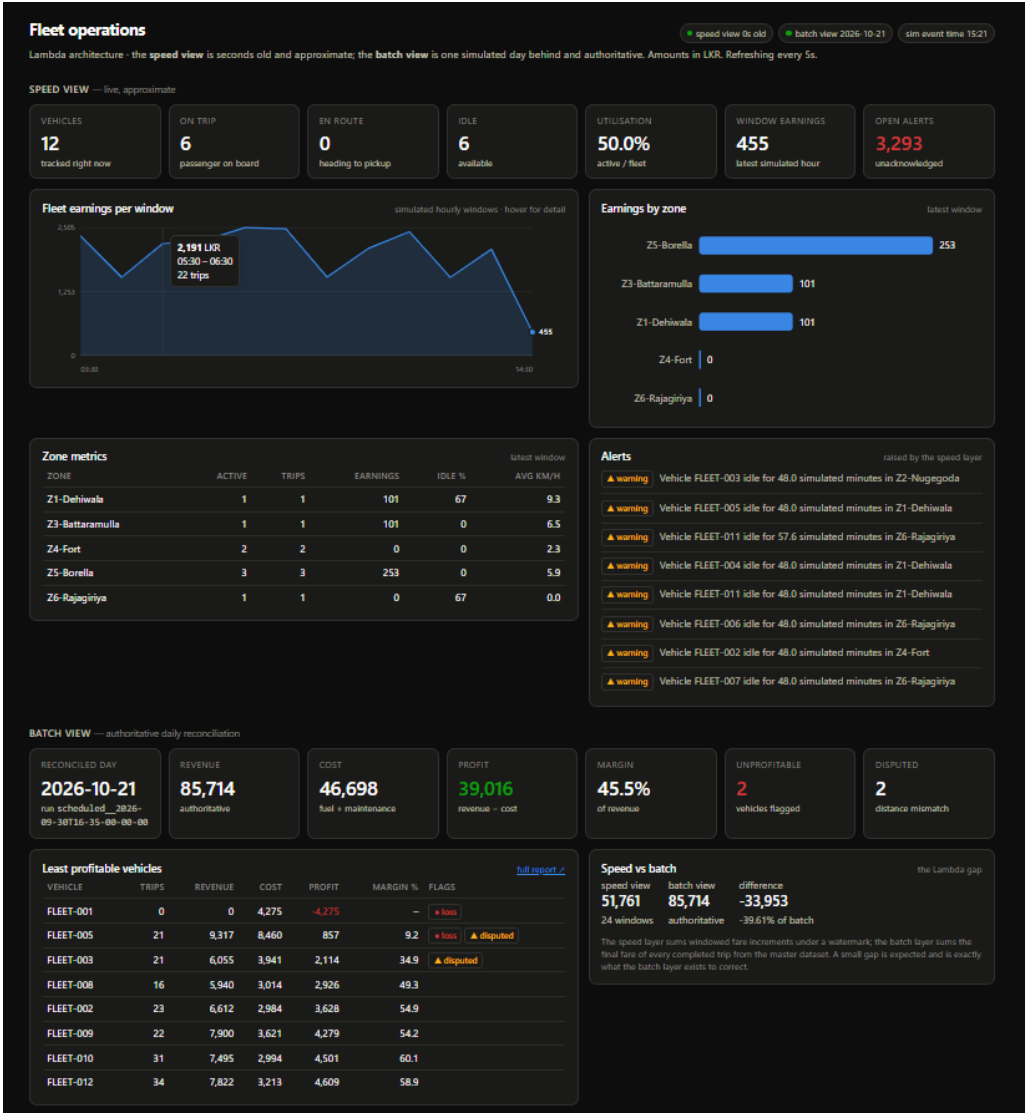


Figure 2: The fleet dashboard showing current vehicle activity and live and daily views. This dashboard image is from a separate run, so its values differ from the 2026-09-29 results below.

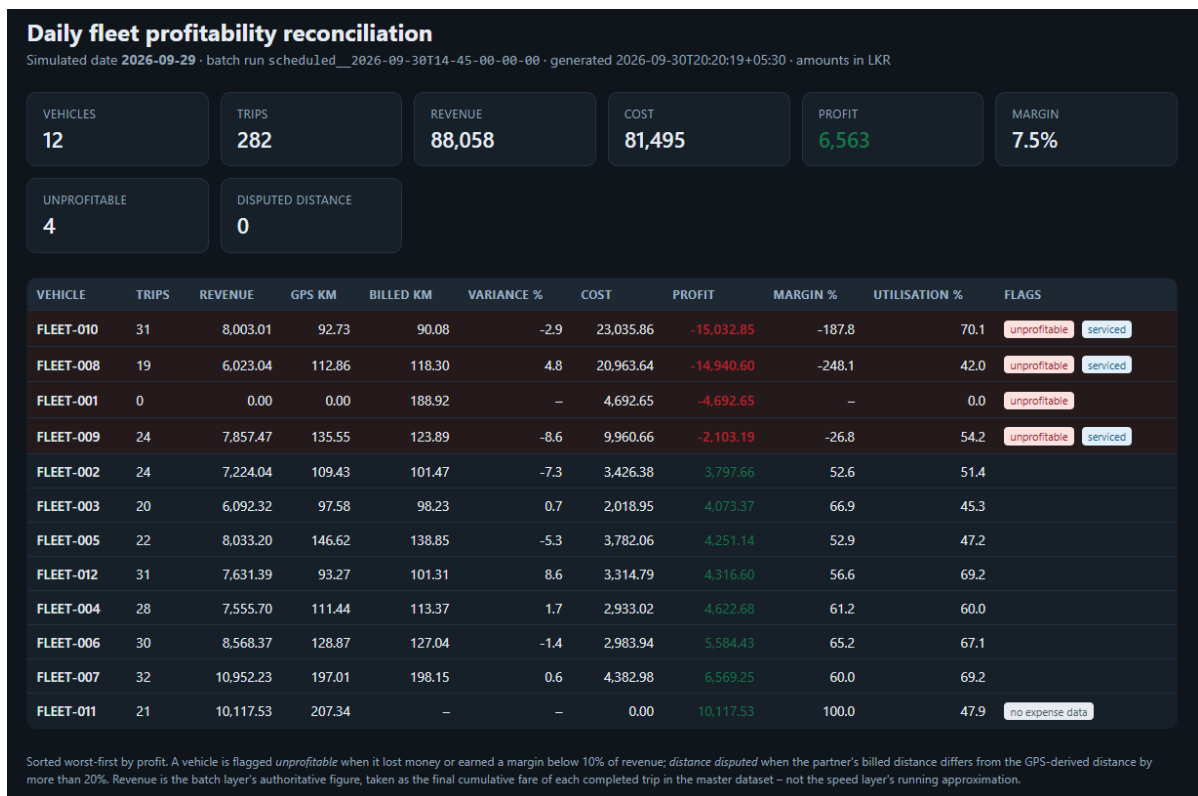


Figure 3: The daily vehicle report showing income, costs, profit or loss, distance, and flags for issues that need review.

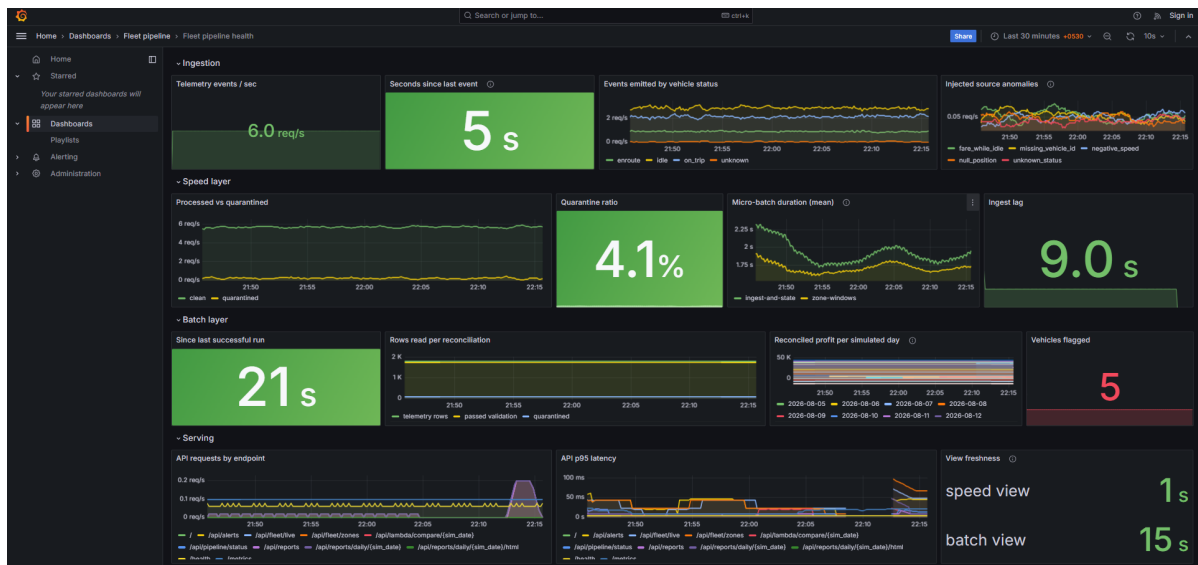


Figure 4: Grafana dashboard showing input rate, data freshness, invalid events, Spark work, daily batch results, and API health.

9 Results

For simulated date 2026-09-29, the daily report showed income of LKR 88,058.30 from 282 completed trips. The live view showed LKR 52,095.79 across 24 time windows. The comparison result showed that the live estimate was LKR 35,962.51 lower, or 40.84 percent below the daily result. The daily report and comparison were generated from the same batch run on 30 September 2026.

Table 5: Example comparison for one simulated day.

View	Result	Meaning
Live earnings	LKR 52,095.79	Estimated from fare changes across 24 windows.
Live trip count	480	Estimated across the live windows.
Daily income	LKR 88,058.30	Calculated from 282 completed trips in the full day.
Difference	LKR -35,962.51	The live estimate was lower than the daily result.
Difference as share of daily income	40.84%	The gap as a share of daily income.

The comparison reports that the two values use different counting methods. The live view adds fare changes in time windows under a watermark, while the daily report adds the final fare for each completed trip in the full event history. The live trip count is an estimate across those windows. The daily trip count counts completed trips for the full day, so it is the better number for a daily report. These results describe one run and may change when the simulated clock moves forward.

10 Limits and Future Improvements

This is a learning project and has limits.

- Spark runs in local mode. The project does not test how a large group of Spark machines handles a failure or shares work.
- Kafka uses one broker. A real service would use several brokers so data remains available if one machine stops.
- Events can be processed again after a restart. The database writes are made safe against repeats, but the Parquet history can contain duplicate rows.
- The Parquet data is not compacted. A long-running system would need to combine many small files or use a table format that manages this work.
- The expense source is simulated. Its costs are not a separate real bill from a fuel company or garage.
- The demo has no user login or encrypted connections. Those protections are needed before using real business or customer data.

At production scale, the team would use a managed or clustered Spark service, several Kafka brokers, protected connections, access control, and alerts sent to an operator. The team would also use a storage format that supports safe changes and cleanup of old data. The expense feed should come from a real approved business source.

11 Conclusion

This project joins live vehicle data with a daily cost file. Kafka and Spark provide a current view. Parquet keeps the history. Airflow starts the daily Spark job when both inputs are ready. PostgreSQL and the API make the results available to the dashboard. Prometheus and Grafana help the team see whether the system is healthy.

The Lambda design fits because the company needs two different answers. Dispatch needs a quick view while vehicles are working. Finance needs a full-day result after costs arrive. The project shows both answers and explains why they can be different.